Dynamic Programming

Edit Distance

Edit Distance

- **Input:** two strings **$X=x_1x_2 \ldots x_i$** and $Y=y_1y_2 \ldots y_j$ where x_i or y_j is a symbol/character
- **Output:** the minimum number of operations (insertions, deletions and replacement) that transforms one string (X) into the other string (y)
  - Measure of similarity between strings

Reading and similar problems

- Readings
  - http://algorithmics.lsi.upc.edu/docs/Dasgupta-Papadimitriou-Vazirani.pdf
    - Chapter 6
- Similar problems
  - https://leetcode.com/problems/edit-distance/description/

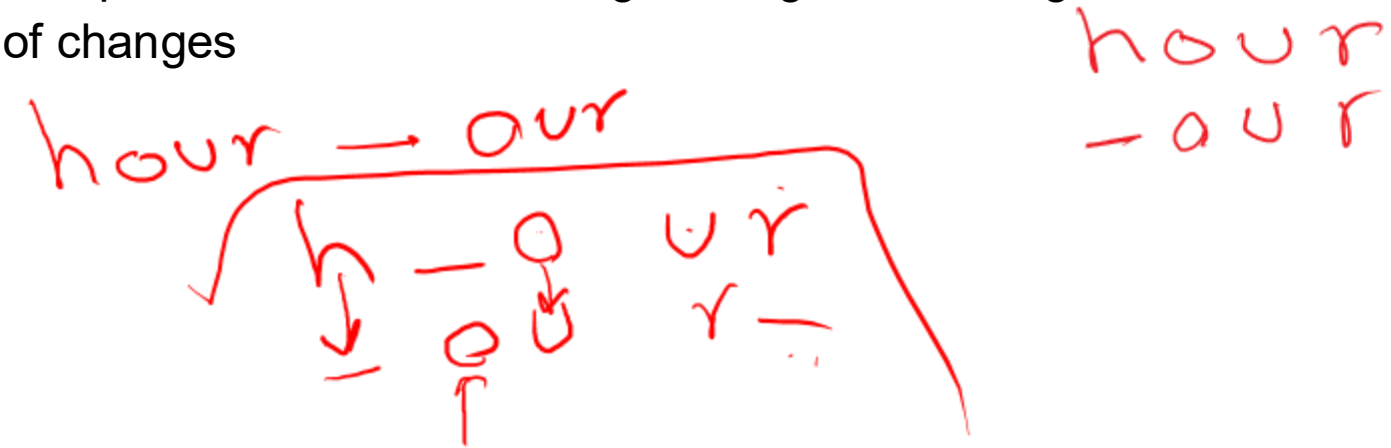
String Edit Operations

- There are three natural types of changes:
  - Replacement – Change a single character from one string to a different character in the other string
    - “shot” to “spot”
  - Insertion – Insert a single character into one string to help it match the other string
    - “ago” to “agog”
  - Deletion – Delete a single character from one string to help it match the other one
    - “hour” to “our”.



String Edit Operations: Visualization

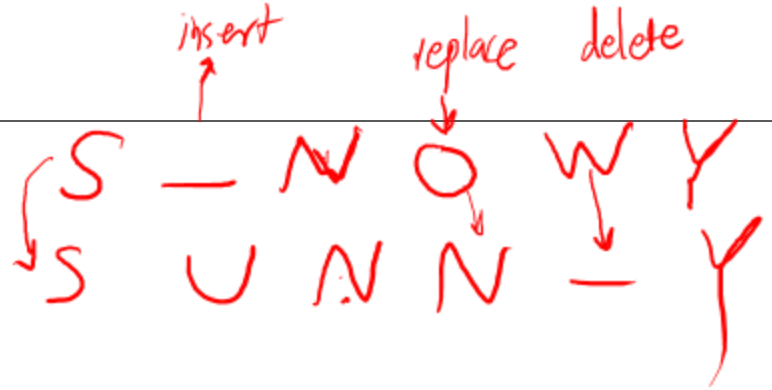
- Visualize editing process by aligning source string above final string
  - **Gaps**: represent insertions and deletions (**insertions** in the top string, **deletion** in bottom)
  - Mismatches: columns with two different characters correspond to **replacement**
  - Match: No cost
  - **Cost of an alignment:** number of gaps + number of mismatch
  - The alignment is not unique
    - Two strings can be aligned in different ways
- The problem turns into finding the alignment that gives the minimum number of changes



Example

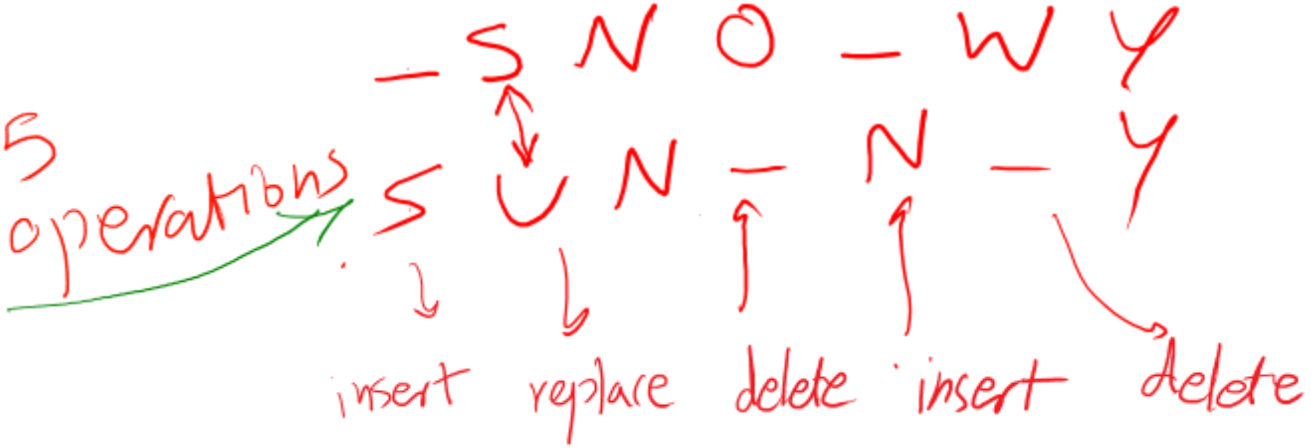
S N O W Y

S U N N Y



3 operations

Edit distance is looking for the minimum number of operations to transform one string into another



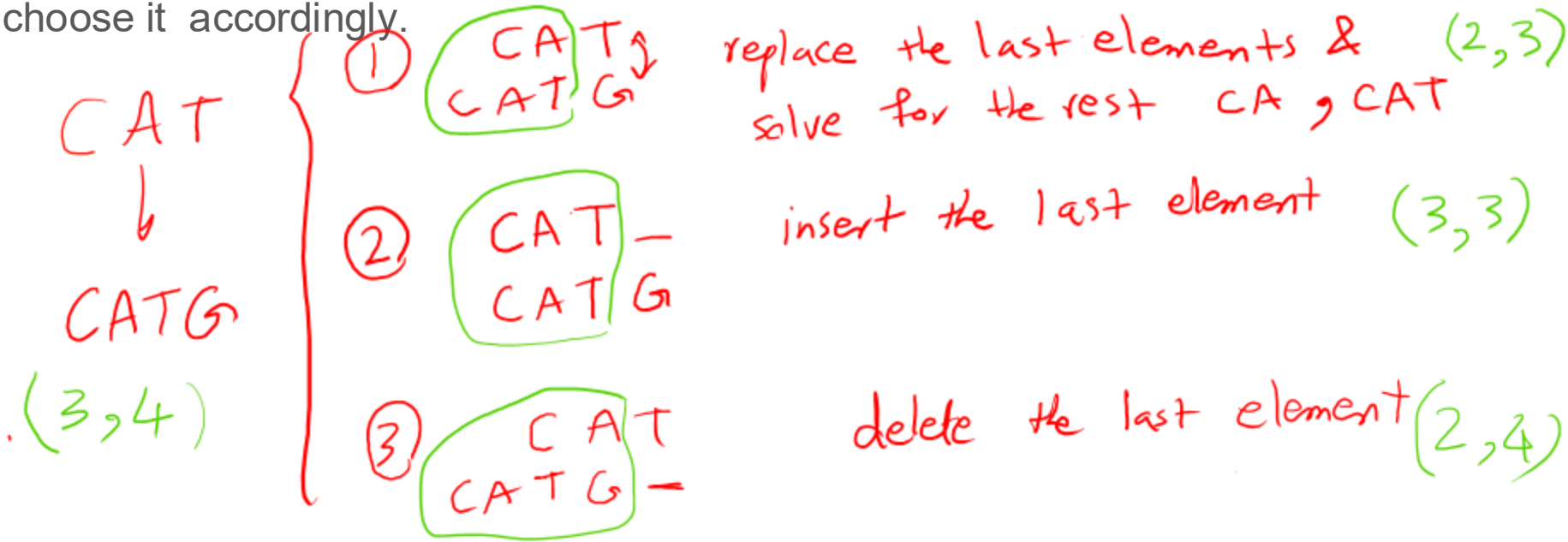
Recursive Algorithm

Idea: the last character in the string needs to be either matched (no cost) or replaced, inserted, or deleted. By knowing the cost of editing the three pairs of smaller strings, we can determine which option provides the optimal solution and choose it accordingly.

Recursive Algorithm

Example CAT → CATG
i = 3 j = 4

Idea: the last character in the string needs to be either matched (no cost) or replaced, inserted, or deleted. By knowing the cost of editing the three pairs of smaller strings, we can determine which option provides the optimal solution and choose it accordingly.



Recursive Formula

- **M(i, j):** Edit distance between **A=x_1x_2 … x_i** to **B=x_1x_2 … x_j**
- Subproblem: edit distance between the prefixes of X and Y
  - Possible choices: match **x_i** to **y_j**, pay **replacement** cost if they are not equal
  - Match **x_i** to blank (delete **x_i**)
  - Match **y_j** to blank (insert **y_j**)
- The solution is in M(n, m)

$$M(i,j) = \begin{cases} M(i-1,j-1) & \text{if } x_i = y_j, \\ \min \begin{cases} 1 + M(i-1,j-1) & \text{replace } x_i \text{ with } y_j \\ 1 + M(i-1,j) & \text{match } x_i \text{ to blank (delete } x_i\text{)} \\ 1 + M(i,j-1) & \text{match } y_j \text{ to blank (insert } y_j\text{)} \end{cases} & \text{if } x_i \neq y_j. \end{cases}$$

Recursive Formula

If the costs of operations are different, the algorithm finds operations with minimum cost
If the costs of operations are the same, the algorithms finds the number of changes

- If r=d=a=1 (count #changes)

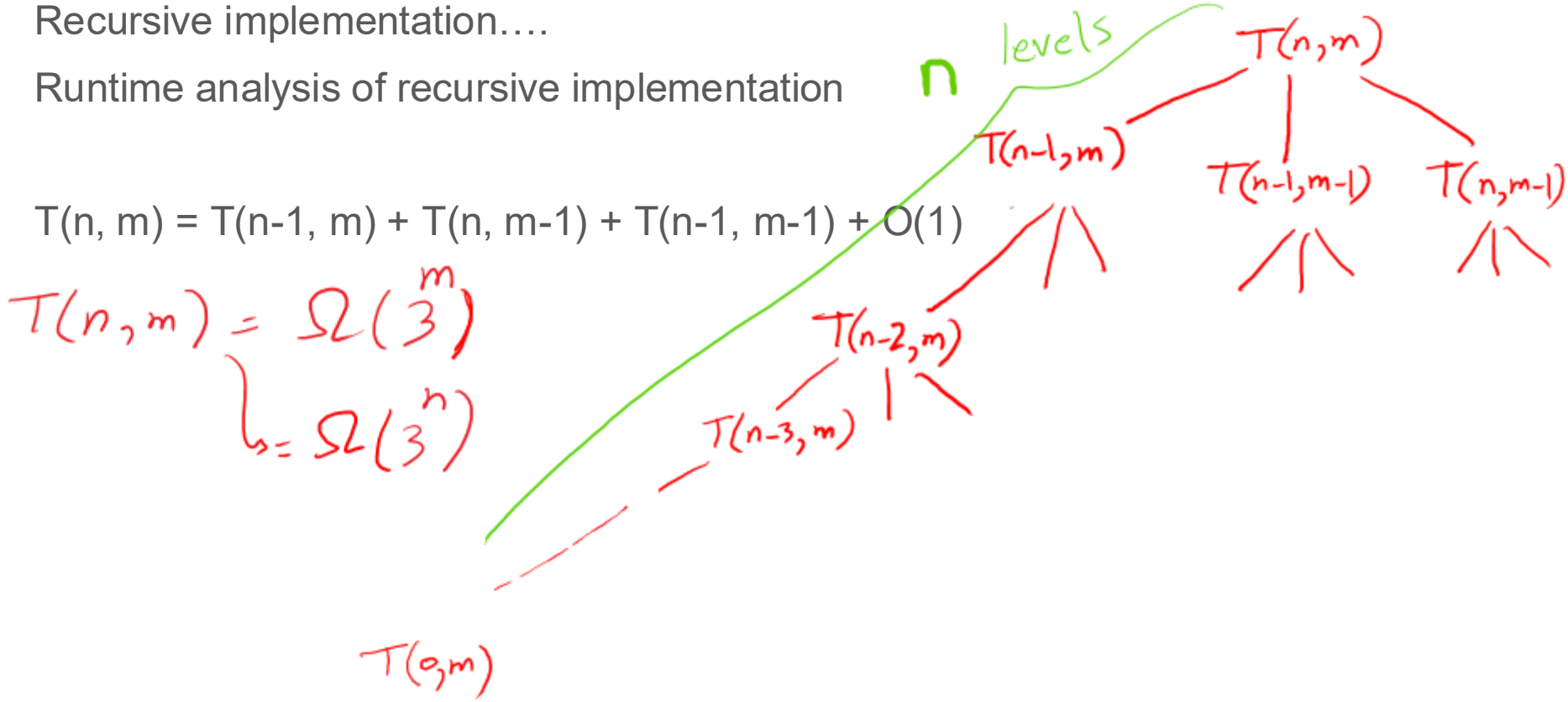
$$M(i,j) = \begin{cases} M(i-1,j-1) & \text{if } x_i = y_j, \\ \min \begin{cases} r + M(i-1,j-1) & \text{replace } x_i \text{ with } y_j \\ d + M(i-1,j) & \text{match } x_i \text{ to blank (delete } x_i\text{)} \\ a + M(i,j-1) & \text{match } y_j \text{ to blank (insert } y_j\text{)} \end{cases} & \text{if } x_i \neq y_j. \end{cases}$$

Challenge

Recursive implementation….

Runtime analysis of recursive implementation

$T(n, m) = T(n-1, m) + T(n, m-1) + T(n-1, m-1) + O(1)$

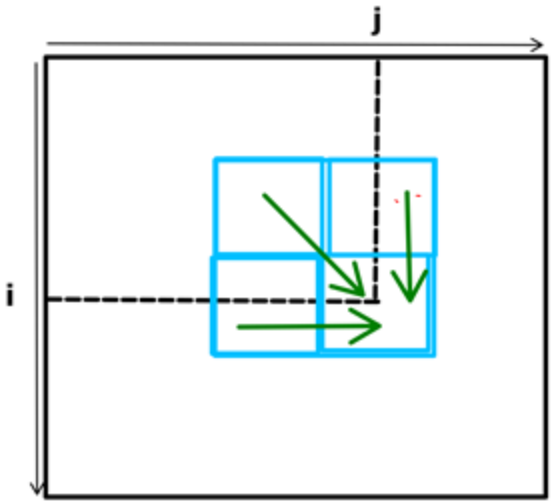


What are the bases cases?

M[i, 0] → i deletes

M[0, j] → j inserts

Bottom-up solution




```
M[0…m, 0…n]
for i = 0 to m
  M[i, 0] = i
for j = 0 to n
  M[0, j] = j
for i from 1 to m
  for j from 1 to n
    if A[i] = B[j]
      M[i, j] = M[i - 1, j - 1]
    else
      M[i, j] = min {
                      M[i-1, j-1] + 1
                      M[i, j-1] + 1
                      M[i-1, j] + 1
                    }
return M[m, n]
```

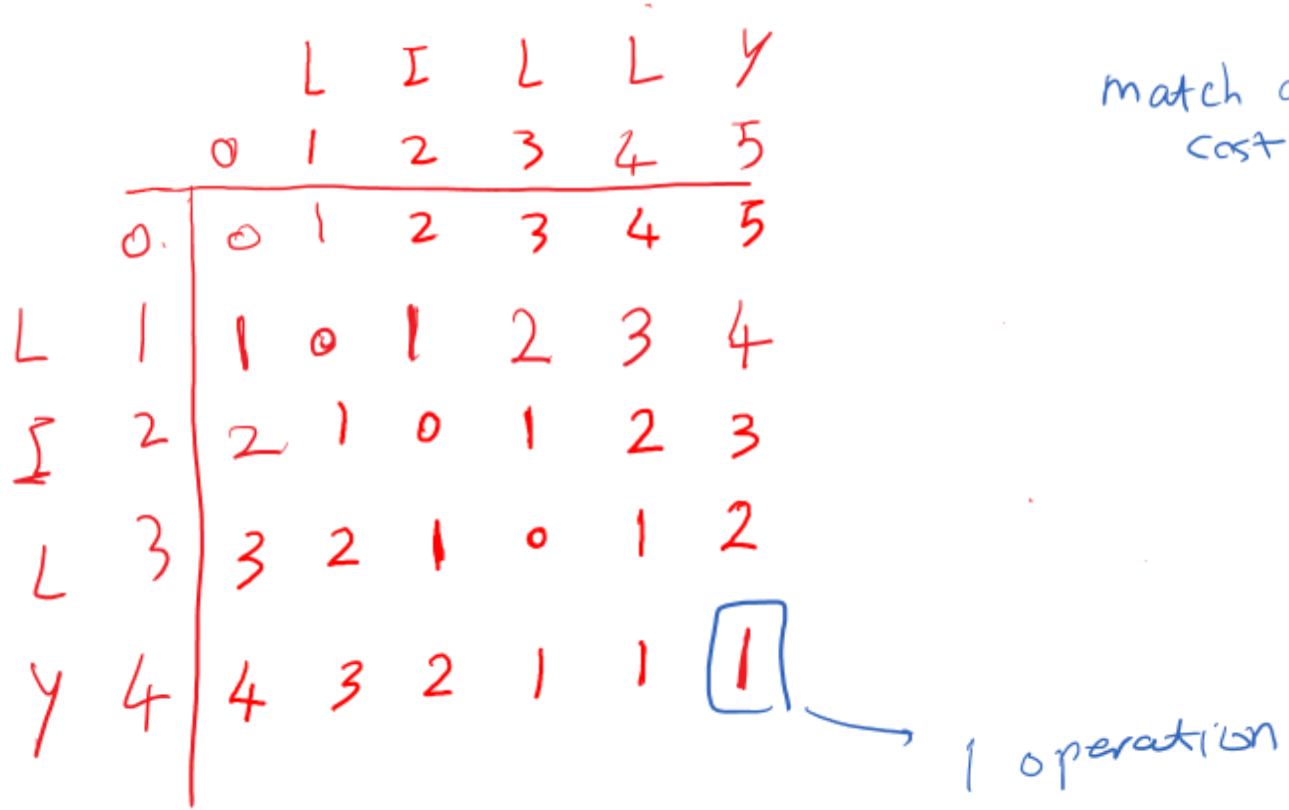
Delete i letters

insert j letters

- **Runtime:**
  - **O(nm) time and O(nm) space**
  - **nm subproblems, constant time each**

Example

LILY ⟶ LILLY



Example LILY → LILLY

